

Watchdog

(DOCUMENTAȚIE)

Universitatea din Bucuresti
Departamentul de Informatica
Instrumente si tehnici de baza in informatica

The Martian: CONSTANTIN ANDREI-GABRIEL
Grupa: 143

0x00) Outline

- ❑ 0x10) Structura proiectului
- ❑ 0x20) Directorul src
- ❑ 0x30)-0x35) Arhitectura proiectului (I)
- ❑ 0x36)-0x39) Arhitectura proiectului (II)
- ❑ 0x40)-0x4B) Implementare – shell
- ❑ 0x50)-0x58) Implementare – python UI
- ❑ 0x60) Idei de îmbunătățiri
- ❑ 0x70) Bibliografie

0x10) Structura proiectului

❑ Watchdog

❑ docs

- ❑ Documentatie.pdf

❑ pdf

- ❑ Prezentare.pdf

❑ src

- ❑ watchdog.sh

- ❑ graphicsDemo.py

- ❑ graphics.py

- ❑ whitelist.txt

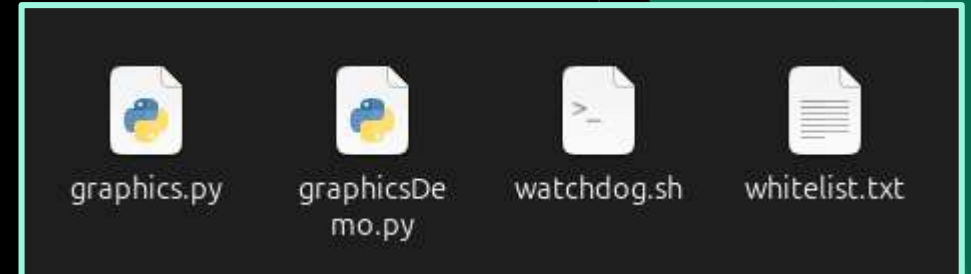
Script
variables

Prompts

Output

0x20) Directorul src

- ❑ În primul rând asigurați-vă că aveți toate fișierele necesare (“watchdog.sh”, “whitelist.txt”, “graphicsDemo.py”, “graphics.py”) în **același director**
- ❑ În al doilea rând, pentru rularea fără erori a UI-ului asigurați-vă că aveți interpretorul de python “python3” și de asemenea librăria “python3-tk” care este folosită de modulul graphics.py
- ❑ `sudo apt-get install python3-tk`



Rularea script-ului va arăta în felul următor:

```
andrei@UbuntuVM:~/Desktop/Watchdog---ITBI-main/Watchdog---ITBI-main$ ./watchdog.sh
CPU LIMIT IS 5%
RAM LIMIT IS 1024 MB
Checked for whitelist.txt file!
WHITELIST:
-- (from input)
gnome
(from whitelist.txt)
Choose what value to sort by [RAM/CPU]CPU
Sorting by CPU USAGE
Do you want to see your processes in Python UI? (y/n)
Process nautilus (PID = 3134) CPU usage exceeded 5% (Used: 30%)
Process 3134 will be stopped (SIGSTOP)
3134 (process ID) old priority 0, new priority 3
Process nautilus (PID = 3134) running after waiting 0 seconds... (SIGCONT)
Process tracker+ (PID = 6988) CPU usage exceeded 5% (Used: 35%)
Process 6988 will be stopped (SIGSTOP)
Process tracker+ (PID = 2646) CPU usage exceeded 5% (Used: 20%)
Process 2646 will be stopped (SIGSTOP)
Process nautilus (PID = 3134) CPU usage exceeded 5% (Used: 14%)
Process 3134 will be stopped (SIGSTOP)
```

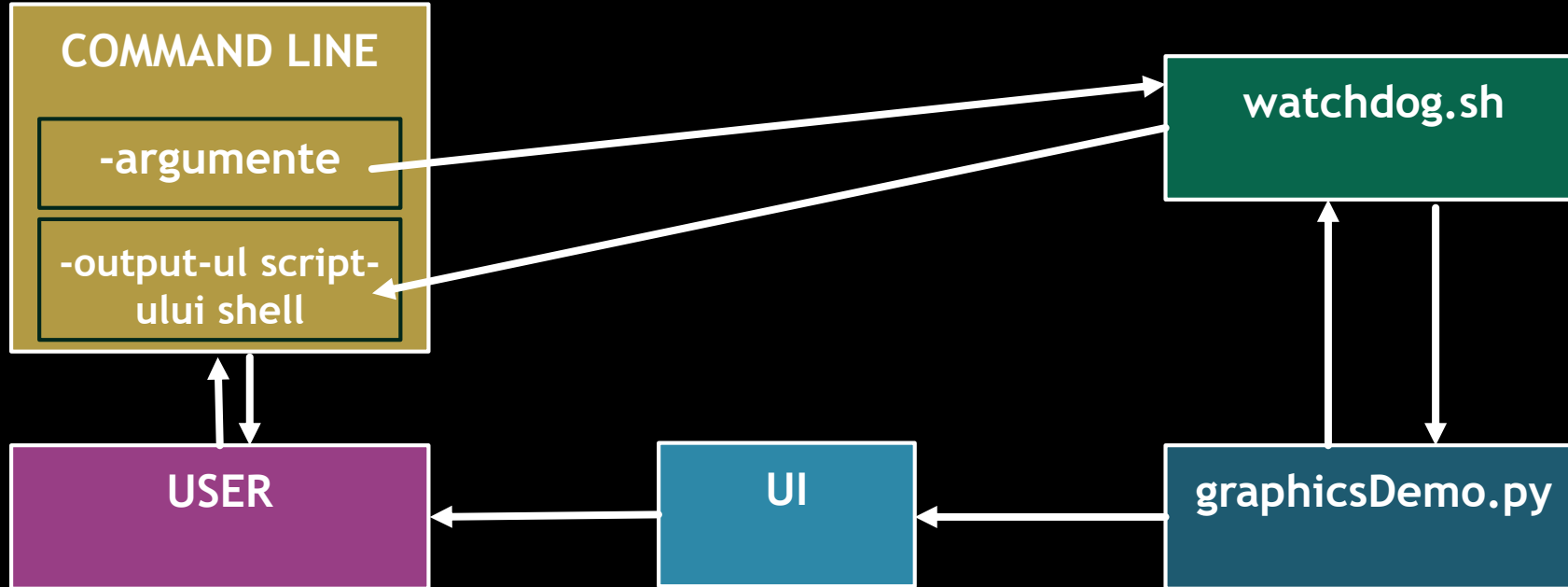
Script variables

Prompts

Output

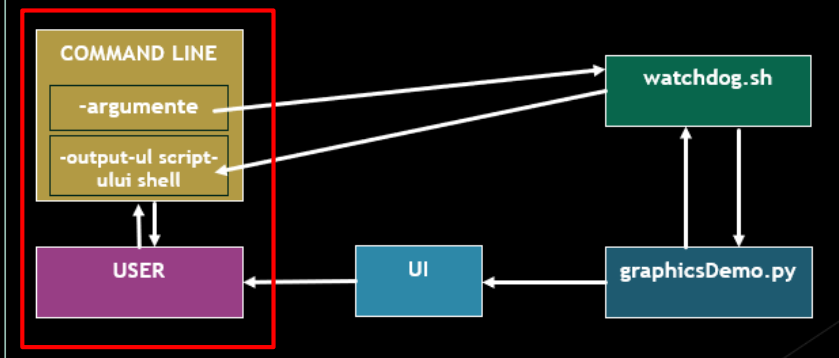
0x30) Arhitectura proiectului (I)

❑ COMUNICAREA ÎNTRE USER ȘI SCRIPT



0x31) Arhitectura proiectului (I)

COMUNICAREA ÎNTRE USER ȘI SCRIPT



- Script-ul are incorporat un mic ghid despre structura comenzii
- (./watchdog.sh --help)
- Avem parametrii [CPU_LIMIT], [RAM_LIMIT] & [*WHITELIST]

```
andrei@UbuntuVM:~/Desktop/Watchdog---ITBI-main./Watchdog---ITBI-main$ ./watchdog.sh --help
----- COMMAND STRUCTURE -----
./watchdog.sh [CPU_LIMIT] [RAM_LIMIT] [*WHITELIST ( KEYWORD_1 KEYWORD_2 ... ) ]

[CPU_LIMIT] --> Maximum workload of CPU (per core) per process, expressed as a percentage
[RAM_LIMIT] --> Maximum physical memory per process, expressed in Megabytes
[*WHITELIST] --> A list of keywords used to exclude processes with matching command names

EXAMPLE: ./watchdog.sh 5 1024 firefox gnome

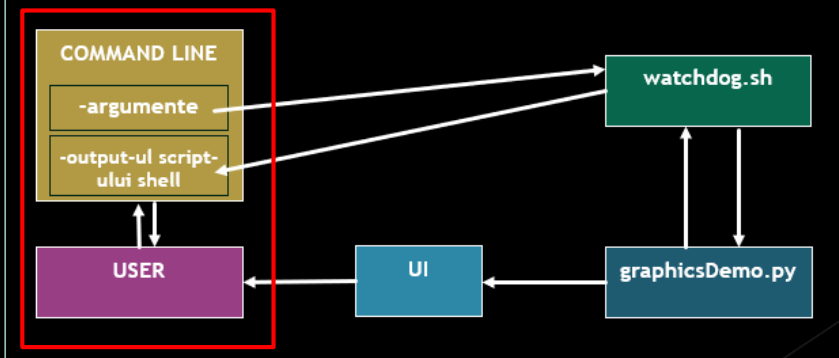
--> All processes using more than 5% of CPU or using more than 1024MB memory (excluding commands matching
"firefox" & "gnome") will be suspended

If the keyword argument NO_COL_ is given, text will print without color. This can be used to redirect the
output of this file to a log file.
```

- 5 <= [CPU_LIMIT]
 - (default: 5)
- 256 <= [RAM_LIMIT]
 - (default: 1024MB)
- WHITELIST – listă de keywords pentru care sunt ignorate procesele cu nume de comenzi similare

0x32) Arhitectura proiectului (I)

COMUNICAREA ÎNTRE USER ȘI SCRIPT



❑ Dacă pasăm ca argument în `*WHITELIST` keyword-ul `"NO_COL_"`, output-ul nu va fi colorat (recomandat pentru pipelineing sau log files)

```
andrei@UbuntuVM:~/Desktop/Watchdog---ITBI-main/Watchdog---ITBI-main$ ./watchdog.sh --help
----- COMMAND STRUCTURE -----

./watchdog.sh [CPU_LIMIT] [RAM_LIMIT] [*WHITELIST ( KEYWORD_1 KEYWORD_2 ... ) ]

[CPU_LIMIT] --> Maximum workload of CPU (per core) per process, expressed as a percentage
[RAM_LIMIT] --> Maximum physical memory per process, expressed in Megabytes
[*WHITELIST] --> A list of keywords used to exclude processes with matching command names

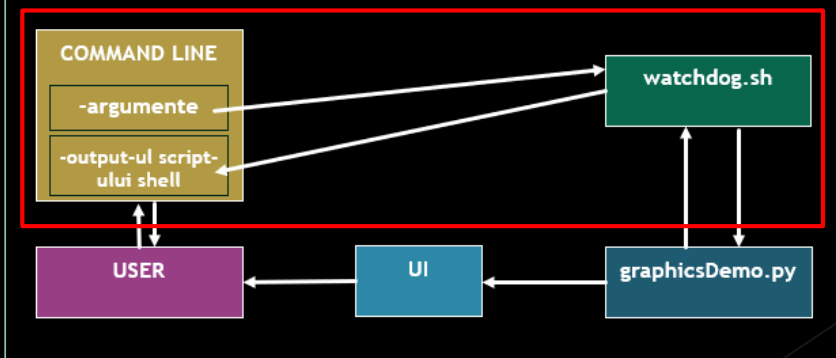
EXAMPLE: ./watchdog.sh 5 1024 firefox gnome

--> All processes using more than 5% of CPU or using more than 1024MB memory (excluding commands matching
"firefox" & "gnome") will be suspended

If the keyword argument NO_COL_ is given, text will print without color. This can be used to redirect the
output of this file to a log file.
```

0x33) Arhitectura proiectului (I)

COMUNICAREA ÎNTRE USER ȘI SCRIPT



```
andrei@UbuntuVM:~/Desktop/Watchdog---ITBI-main/Watchdog---ITBI-main$ ./watchdog.sh
CPU LIMIT IS 5%
RAM LIMIT IS 1024 MB
Checked for whitelist.txt file!
WHITELIST:
- (from input)
  gnome
- (from whitelist.txt)
Choose what value to sort by [RAM/CPU]CPU
Sorting by CPU USAGE
Do you want to see your processes in Python UI? (y/n)
Process nautilus (PID = 3134) CPU usage exceeded 5% (Used: 30%)
Process 3134 will be stopped (SIGSTOP)
3134 (process ID) old priority 0, new priority 3
Process nautilus (PID = 3134) running after waiting 0 seconds... (SIGCONT)
Process tracker+ (PID = 6988) CPU usage exceeded 5% (Used: 35%)
Process 6988 will be stopped (SIGSTOP)
Process tracker+ (PID = 2646) CPU usage exceeded 5% (Used: 20%)
Process 2646 will be stopped (SIGSTOP)
Process nautilus (PID = 3134) CPU usage exceeded 5% (Used: 14%)
Process 3134 will be stopped (SIGSTOP)
```

Script
variables

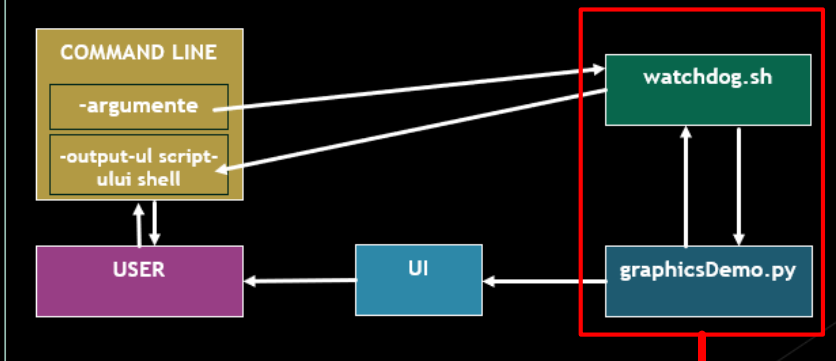
Prompts

Output

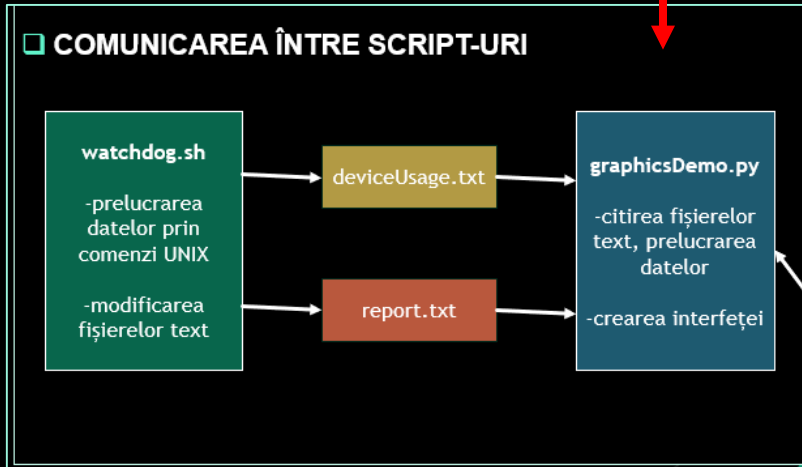
- Argumentele sunt preluate în script, iar pragurile sunt stabilite cum am menționat anterior
- Ca output, script-ul va afișa în terminal, după configurarea parametrilor și funcțiunilor, procesele care au depășit pragul, menționând numele comenzii, Process ID-ul și cantitatea de resurse folosite care au încălcat limitele stabilite
- De asemenea, se va afișa kill signal-ul ales de script (SIGSTOP, în cazul pragului CPU, respectiv SIGTERM, în cazul pragului RAM)
- După un anumit interval de timp, când procesorul e mai disponibil, procesul poate rula în continuare (SIGCONT), și ni se afișează wait-time-ul acestuia

0x34) Arhitectura proiectului (I)

COMUNICAREA ÎNTRE USER ȘI SCRIPT



COMUNICAREA ÎNTRE SCRIPT-URI

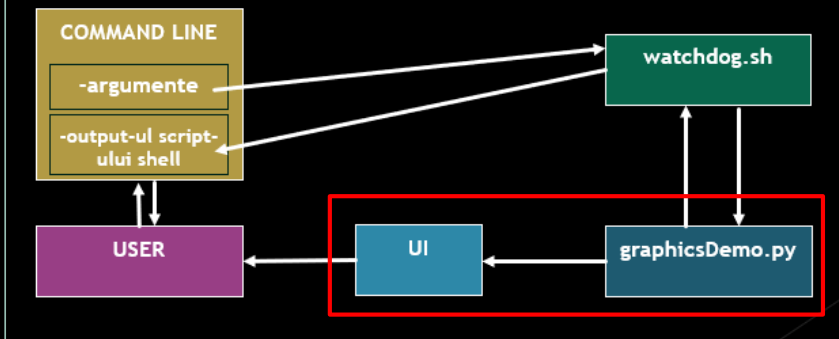


Comunicarea între script-ul shell și script-ul python, se va face, cum vom observa la partea II, prin sistemul de fișiere

Script-ul shell va suprascrive aceste fișiere la un interval de timp, iar script-ul python va citi aceste fișiere la un interval de timp

0x35) Arhitectura proiectului (I)

COMUNICAREA ÎNTRE USER ȘI SCRIPT

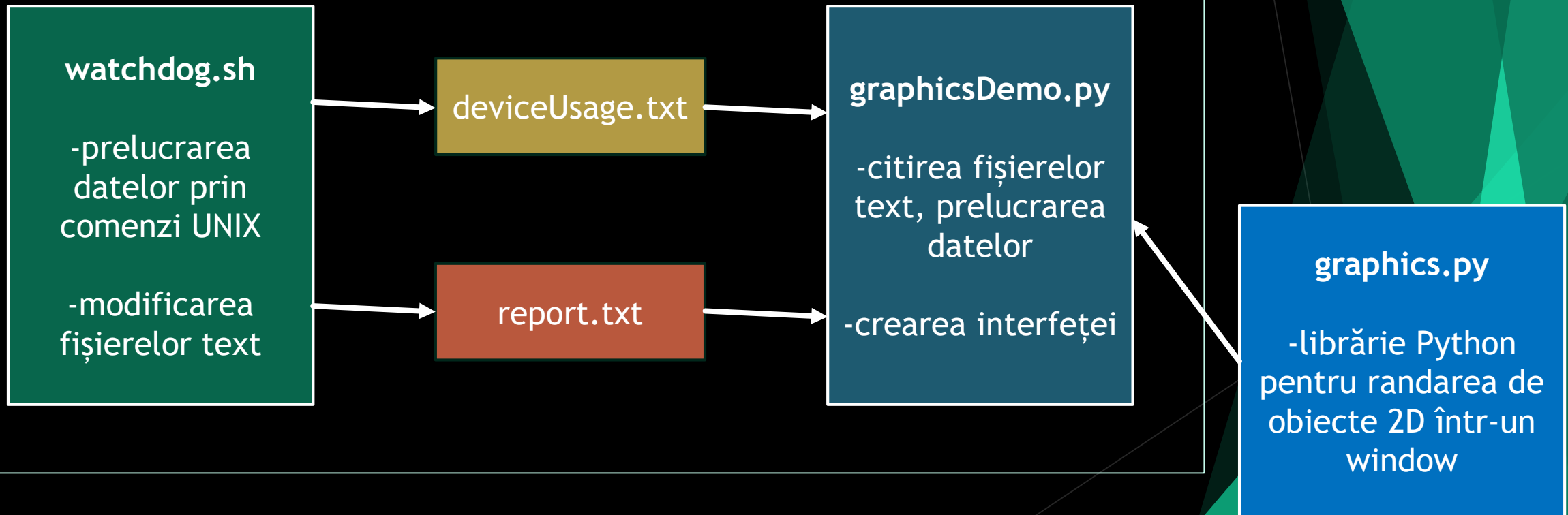


Script-ul Python este destul de complex, folosind script-ul extern `graphics.py` (<https://mcsp.wartburg.edu/zelle/python/>)

- Script-ul `graphicsDemo.py` are următoarele responsabilități:
 - Prelucrarea datelor din fișierele accesate
 - Crearea tabeli de procese și actualizarea ei
 - Crearea progress bar-urilor pentru RAM și CPU
 - Crearea graficelor

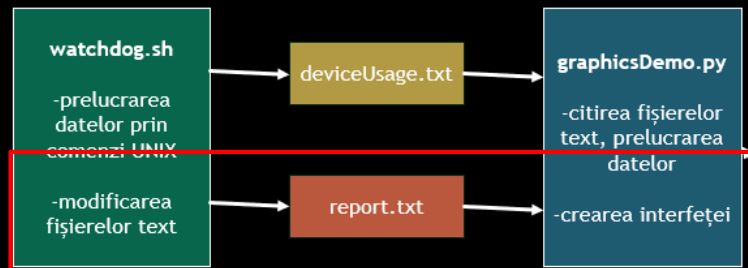
0x36) Arhitectura proiectului (II)

□ COMUNICAREA ÎNTRE SCRIPT-URI



0x37) Arhitectura proiectului (II)

COMUNICAREA ÎNTRE SCRIPT-URI



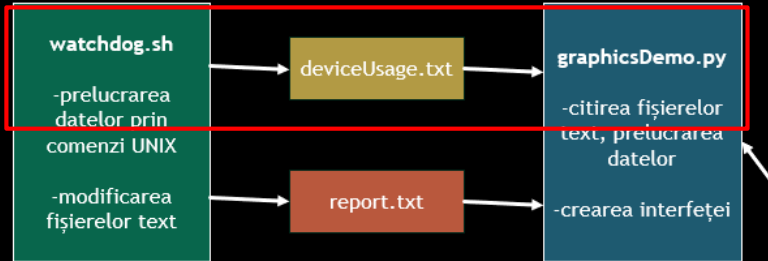
- ❑ În tabela proceselor totuși, vom avea mai puține coloane, excluzând memoria virtuală, shared și PR și NI (adică prioritatea procesului)

- ❑ Fișierul report.txt reprezintă unul din cele 2 medii de comunicare ale celor 2 script-uri

- ❑ Acesta conține output-ul comenzii top și este necesar ca script-ul python și cel shell să aibă toate field-urile în aceeași ordine

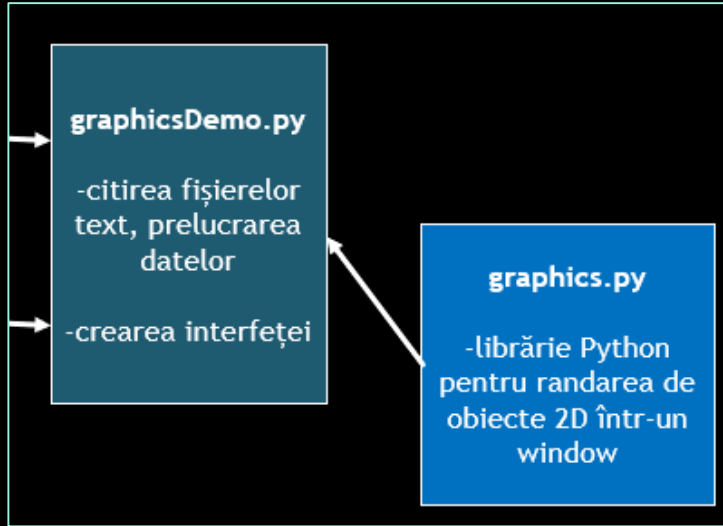
0x38) Arhitectura proiectului (II)

COMUNICAREA ÎNTRE SCRIPT-URI



- Fișierul **deviceUsage.txt** este updatat de script-ul shell calculând cât de mult sunt utilizate procesorul și memoria fizică, dar și cea shared între programe.
- Script-ul python salvează aceste informații într-o listă de lungime **K** pentru a crea ulterior graficele și progress-bar-urile

0x39) Arhitectura proiectului (II)



- ❑ **Fișierul `graphics.py` este responsabil pentru reprezentarea în window a elementelor geometrice**
- ❑ Acest script extern se folosește de programarea orientată pe obiecte pentru a realiza metode și proprietăți intuitive pentru figuri geometrice
- ❑ Câteva dintre funcțiile acestui modul sunt:
 - ❑ `Point(x,y)` -> obiect de tip punct
 - ❑ `Rectangle(Point(x1,y1),Point(x2,y2))` -> obiect de tip dreptunghi
 - ❑ `Polygon(Point(x1,y1),Point(x2,y2),...)` -> obiect de tip polygon
 - ❑ `Text(x,y,str)` -> obiect de tip text centrat la punctul (x,y)

0x40) Implementare - shell

- ❑ Abordarea prototipului:
 - ❑ Creăm un while loop care să execute o dată la o secundă comanda top
 - ❑ Vom folosi comanda top care va rula o singură dată (flag -n 1) în batch mode (flag -b) per iterație
 - ❑ Salvăm output-ul acestei comenzi într-un fisier text și iterăm apoi linie cu linie
 - ❑ (linie din fisier ⇔ informații despre process: PID, USER, %MEM, %CPU, RES, TIME, COMMAND etc.)
 - ❑ Verificăm pentru fiecare linie:
 - ❑ (memoria residentă) RES > 1024 (MiB) (adică 1 GiB)
 - ❑ (utilizarea procesorului) %CPU > 5.0
 - ❑ Dacă una din condiții este îndeplinită, omorâm procesul (comanda kill)

0x41) Implementare – shell

- ❑ Înainte de toate, am definit câteva variabile care rețin coduri pentru diferite culori, pe care le vom folosi în contextul comenzii echo
- ❑ Flag-ul -e ne permite să utilizăm caractere speciale, în cazul nostru \n (newline) și \e[Xm, unde X este un cod de culoare
- ❑ Astfel putem crea un fel de Command Line Interface (CLI)
- ❑ Totodată aici putem defini un flag custom (--help) care să ne dea informații despre structura comenzii
- ❑ Reamintim că script-ul ia ca parametrii: CPU_LIMIT, RAM_LIMIT și *keywords (pentru excluderea anumitor procese)
- ❑ Putem să adăugăm în *keywords și cuvântul-cheie NO_COL_ care va elimina culoarea textului (spre exemplu vrem să redirecționăm output-ul comenzii într-un log file fără alte caractere de tip culoare text)

```
#!/bin/bash

#color
r="\e[31m" #red
g="\e[32m" #green
b="\e[34m" #blue
y="\e[33m" #yellow
p="\e[35m" #pink
en="\e[0m" #end color

HELP=$1
if [[ $HELP == "--help" ]]
then
    echo -e "----- COMMAND STRUCTURE -----"
    echo -e "$p./watchdog.sh [CPU_LIMIT] [RAM_LIMIT]$en"
    echo -e "\n $p[CPU_LIMIT]$en --> Maximum workload o"
    echo -e "\n $p[RAM_LIMIT]$en --> Maximum physical m"
    echo -e "\n $p[*WHITELIST]$en --> A list of $p keywo"
    names"
    echo -e "\n \n EXAMPLE: ./watchdog.sh 5 1024 firefo"
    echo -e "--> All processes using more than 5% of CP"
    matching "firefox" & "gnome") will be suspended \n"
    echo -e "If the keyword argument NO_COL_ is given,"
    the output of this file to a log file. \n"
    exit
fi
```

0x41) Implementare – shell

- ❑ Mai întâi, determinăm pragurile impuse date ca input când rulăm comanda ./watchdog.sh
- ❑ Acestea se citesc în următoarea ordine:
 - ❑ 1) CPU LIMIT (%)
 - ❑ 2) RAM LIMIT (MB)
- ❑ Dacă una din aceste valori (sau ambele) nu sunt date ca input, atunci setăm valorile default:
 - ❑ CPU LIMIT = 5%
 - ❑ RAM LIMIT = 1024 MB (1 GB)
- ❑ De asemenea, verificăm dacă limita impusă pentru memorie este mai mare decât 256 MB și cea pentru CPU este mai mare decât 5%. Am ales aceste minime pentru a evita închiderea prea multor procese
- ❑ Anunțăm utilizatorul în Command Line pragurile impuse

```
cpuLimit=$1
ramLimitMB=$2
keywords=${@:3}

echo $keywords | grep -q "NO_COL_" && r="" && g="" && b="" && y="" && p="" && en=""

if [ -z $cpuLimit ]
then
    cpuLimit=5
else
    if [[ $cpuLimit < 5 ]]
    then
        echo -e "$r CPU LIMIT TOO LOW (<5%) $en"
        exit
    fi
fi

if [ -z $ramLimitMB ]
then
    ramLimitMB=1024
else
    if [[ $ramLimitMB < 256 ]]
    then
        echo -e "$r RAM LIMIT TOO LOW (<256MB) $en"
        exit
    fi
fi

echo -e "$p CPU LIMIT IS $cpuLimit% $en"
echo -e "$p RAM LIMIT IS $ramLimitMB MB $en"
```

0x42) Implementare – shell

- ❑ Apoi, transformăm valoarea RAM în KB (pentru că sub forma asta primim datele folosind comanda top*)
 - ❑ (Când folim **arithmetic expansion**** NU prefixăm variabilele cu \$)
- ❑ Adăugăm o funcție care verifică dacă fișierele necesare există în directorul proiectului
- ❑ Arătăm cuvintele din whitelist (*keywords + conținut whitelist.txt)

```
KB=$(( ramLimitMB << 10 ))

checkFileExists() {
    if [ -f $1 ]
    then
        echo "Checked for $1 file!"
    else
        echo "File $1 does not exist in the current directory!"
        exit
    fi
}

if [ -f stall.txt ]
then
    rm stall.txt
fi

checkFileExists "whitelist.txt"

echo -e "WHITELIST: \n $p-- $keywords$en (from input) \n $p'cat whitelist.txt'$en \n(from whitelist.txt)"
```

* “RES: How much physical RAM the process is using, measured in kilobytes.”

<https://www.geeksforgeeks.org/linux-unix/top-command-in-linux-with-examples/>

** “Integer constants follow the C language definition, without suffixes or character constants.”

https://www.gnu.org/software/bash/manual/html_node/Shell-Arithmetic.html

0x43) Implementare – shell

- ❑ Apoi, utilizatorul va primi un prompt în care va alege un criteriu în baza căruia vom sorta procesele
- ❑ Sortarea proceselor se poate face automat folosind flag-ul -o când executăm comanda top
- ❑ Totodată, trebuie să modificăm valoarea variabilei SORTBY ca să fie în concordanță cu field-urile comenzii top
 - ❑ “CPU” ⇔ “%CPU”
 - ❑ “RAM” ⇔ “RES”

```
if [[ $SORTBY == "RAM" || ($SORTBY != "RAM" && $SORTBY != "CPU") ]]
then
    SORTBY="RES"
    echo "Sorting by RAM USAGE"
fi
if [[ $SORTBY == "CPU" ]]
then
    SORTBY="%CPU"
    echo "Sorting by CPU USAGE"
fi

testPID() {
    if [ -d "/proc/$1" ]
    then
        return 0
    else
        #echo -e "\e[33m Process $2 with ID $1 NOT FOUND IN /proc $en"
        return 1
    fi
}

read -p "Do you want to see your processes in Python UI? (y/n)" yn

touch "report.txt"
touch "deviceUsage.txt"
```

0x43) Implementare – shell

- ❑ Definim funcția testPID(), care verifică dacă există procesul în /proc
- ❑ Kernel-ul atribuie fiecărui proces un director special, conținând informații despre acesta, în directorul /proc
- ❑ Exit code-ul funcției va fi folosit în multe condiții, 0 desemnând “true” și 1 desemnând “false”
- ❑ Apoi întrebăm utilizatorul dacă dorește să afișeze UI-ul și creăm fișierele necesare pentru comunicarea între script-ul shell și script-ul Python

```
if [[ $SORTBY == "RAM" ]] || ($SORTBY != "RAM" && $SORTBY != "CPU") ]]
then
    SORTBY="RES"
    echo "Sorting by RAM USAGE"
fi
if [[ $SORTBY == "CPU" ]]
then
    SORTBY="%CPU"
    echo "Sorting by CPU USAGE"
fi

testPID() {
    if [ -d "/proc/$1" ]
    then
        return 0
    else
        #echo -e "\e[33m Process $2 with ID $1 NOT FOUND IN /proc $en"
        return 1
    fi
}

read -p "Do you want to see your processes in Python UI? (y/n)" yn

touch "report.txt"
touch "deviceUsage.txt"
```

0x44) Implementare – shell

- ❑ Acum vom întreba utilizatorul dacă dorește să deschidă interfața UI, iar dacă răspunsul este pozitiv, verificăm că există biblioteca “graphics.py” și programul în sine care crează interfața “graphicsDemo.py”
- ❑ Executăm programul Python folosind interpretorul python3 și punem acest nou proces creat în background

```
read -p "Do you want to see your processes in Python UI? (y/n)" yn

touch "report.txt"
touch "deviceUsage.txt"

if [[ $yn == "y" ]]
then
    checkFileExists "graphics.py"
    checkFileExists "graphicsDemo.py"
    python3 graphicsDemo.py &
    PYTHON_PID=$!
    echo -e "$p Initializing window (PID=$PYTHON_PID)$en"
    trap "echo -e ' \n WATCHDOG TERMINATED' ; kill $PYTHON_PID ; rm report.txt ; rm deviceUsage.txt ; exit" SIGINT SIGTERM
    sleep 2
    echo -e "$p Initialization done$en"
else
    trap "echo -e ' \n WATCHDOG TERMINATED' ; rm report.txt ; rm deviceUsage.txt ; exit" SIGINT SIGTERM
fi
```

0x44) Implementare – shell

Aici ne întâlnim cu o **problemă**: închiderea script-ului shell nu închide programul Python

- ❑ Soluția găsită:
 - ❑ Salvăm **PID-ul** procesului care tocmai ce a fost creat anterior (python3)
 - ❑ Folosind comanda **trap** vom executa **comenzile subliniate** fix înaintea unui semnal **SIGINT** sau **SIGTERM** (adică fix înainte de închiderea script-ului shell)

```
read -p "Do you want to see your processes in Python UI? (y/n)" yn

touch "report.txt"
touch "deviceUsage.txt"

if [[ $yn == "y" ]]
then
    checkFileExists "graphics.py"
    checkFileExists "graphicsDemo.py"
    python3 graphicsDemo.py &
    PYTHON_PID=$!
    echo -e "$p Initializing window (PID=$PYTHON_PID)$en"
    trap "echo -e ' \n WATCHDOG TERMINATED' ; kill $PYTHON_PID ; rm report.txt ; rm deviceUsage.txt ; exit" SIGINT SIGTERM
    sleep 2
    echo -e "$p Initialization done$en"
else
    trap "echo -e ' \n WATCHDOG TERMINATED' ; rm report.txt ; rm deviceUsage.txt ; exit" SIGINT SIGTERM
fi
```

0x45) Implementare – shell

- ❑ Acum vrem să definim o funcție care va scrie în fișierul “deviceUsage.txt” memoria folosită (totală și shared) și utilizarea procesorului sub formă de procent
- ❑ Această funcție va fi folosită pentru graficul interfeței UI care monitorizează în timp aceste valori (trebuie preluate de program-ul Python)

```
getDeviceUsage() {  
    CPU_USAGE=$( top -b -n 1 | head -n +5 | grep "Cpu(s)" | awk '{print 100 - $8}' )  
    FREE_RAM=$(free --mega | tail +2 | head -1)  
    echo $FREE_RAM | while read MEM TOTAL USED FREE SHARED CACHE AVAILABLE  
    do  
        P_USED=$(( $USED * 100 / $TOTAL ))  
        P_SHR=$(( $SHARED * 100 / $TOTAL ))  
        echo -e "\n $CPU_USAGE $P_USED $P_SHR $TOTAL" > deviceUsage.txt  
    done  
}  
  
USER=$(whoami)
```

Putem optimiza prin citirea fișierelor din /proc

0x46) Implementare – shell

- ❑ Optimizare (în versiunea 1.22 a proiectului)
- ❑ Vom folosi comenzile într-o buclă, deci trebuie să ne gândim la optimizarea timpului de rulare al acestei comenzi
- ❑ **Folosim comanda time pentru a compara cele 2 abordări**

```
andrei@UbuntuVM:~/Desktop/Watchdog---ITBI-main. (1)/Watchdog---ITBI-main$ time top  
-bn 1 | head -n +5 | grep "Cpu(s)" | awk '{print( 100 - $8 )}'  
5.8
```

```
real    0m0.406s  
user    0m0.027s  
sys     0m0.042s
```

```
andrei@UbuntuVM:~/Desktop/Watchdog---ITBI-main. (1)/Watchdog---ITBI-main$ time read  
-rs cpustxt user nice system idle iowait irq softirq x0 x1 x2 < /proc/stat
```

```
real    0m0.000s  
user    0m0.000s  
sys     0m0.000s
```

Această abordare este mult mai eficientă

```
andrei@UbuntuVM:~/Desktop/Watchdog---ITBI-main. (1)/Watchdog---ITBI-main$
```

0x46) Implementare – shell

- ❑ Totuși, comanda ne oferă informații destul de “crude”, și informații despre acestea găsim aici: <https://www.linuxhowtos.org/System/procstat.htm>

```
user nice system idle iowait irq softirq x0 x1 x2 < /proc/stat
```

These numbers identify the amount of time the CPU has spent performing different kinds of work. Time units are in USER_HZ or Jiffies (typically hundredths of a second).

The meanings of the columns are as follows, from left to right:

- user: normal processes executing in user mode
- nice: niced processes executing in user mode
- system: processes executing in kernel mode
- idle: twiddling thumbs
- iowait: waiting for I/O to complete
- irq: servicing interrupts
- softirq: servicing softirqs

- ❑ Observăm variabila IDLE și dacă o împărțim la suma variabilelor, obținem cât la sută din timp procesorul a fost idle

0x46) Implementare – shell

- ❑ Ca să aflăm procentul de utilizare al procesorului, trebuie să raportăm rezultatul la un interval de timp
- ❑ Între $t=0s$ și $t=3s$, procesorul a fost idle $x\%$ din timp
- ❑ Calculăm cu formula $(idle2 - idle1) / (sum2 - sum1)$, valorile fiind cumulative de la boot
- ❑ “The CPU usage can be measured over an interval of time only. This means we have to read the values from `/proc/stat` on a fixed interval and calculate the delta from these readings.” (<https://www.idnt.net/en-GB/kb/941772>)

```
andrei@UbuntuVM:~/Desktop$ head -n 2 /proc/stat && sleep 3 && echo "after 3s" &&
head -n 2 /proc/stat
cpu 3817 63 2058 73727 490 0 332 0 0 0
cpu0 1004 25 569 18122 122 0 41 0 0 0
after 3s
cpu 3823 63 2061 74938 490 0 333 0 0 0
cpu0 1006 25 570 18421 122 0 41 0 0 0
andrei@UbuntuVM:~/Desktop$ echo "3817+63+2058+73727+490+0+332+0+0+0" | bc
80487
andrei@UbuntuVM:~/Desktop$ echo "3823+63+2061+74938+490+333" | bc
81708
andrei@UbuntuVM:~/Desktop$ echo "81708 - 80487" | bc && echo " | diferenta dintre
sume"
1221
| diferenta dintre sume
andrei@UbuntuVM:~/Desktop$ echo "74938 - 73727" | bc && echo " | diferenta dintre
idle"
1211
| diferenta dintre idle
andrei@UbuntuVM:~/Desktop$ echo " 1211 / 1221 * 100 " | bc
0
andrei@UbuntuVM:~/Desktop$ echo " 1211 * 100 / 1221 " | bc
99
andrei@UbuntuVM:~/Desktop$ deci 99% idle, adica 1% utilizare CPU
```

0x46) Implementare – shell

- ❑ Ca să aflăm procentul de utilizare al procesorului, trebuie să raportăm rezultatul la un interval de timp
- ❑ Între $t=0s$ și $t=3s$, procesorul a fost idle $x\%$ din timp

```
getDeviceUsage() {  
    read -rs cpustxt user nice system idle iowait irq softirq x0 x1 x2 < /proc/stat  
    sum1=$(( user + nice + system + idle + iowait + irq + softirq + x0 + x1 + x2 ))  
    idle1=$idle  
    #sleep 3  
    for i in 1-12  
    do  
        read -t 0.25 -n 1 key && touch stall.txt && read -p " | Program stalled! Press Enter to continue: " && rm stall.txt  
    done  
    read -rs cpustxt user nice system idle iowait irq softirq x0 x1 x2 < /proc/stat  
    sum2=$(( user + nice + system + idle + iowait + irq + softirq + x0 + x1 + x2 ))  
    idle2=$idle  
    idleTotal=$(( idle2 * 100 - idle1 * 100 ))  
    sumTotal=$(( sum2 - sum1 ))  
    CPU_USAGE=$(( 100 - idleTotal / sumTotal ))  
    free --mega | tail +2 | head -1 | while read MEM TOTAL USED FREE SHARED CACHE AVAILABLE  
    do  
        P_USED=$(( USED * 100 / TOTAL ))  
        P_SHR=$(( SHARED * 100 / TOTAL ))  
        echo -e "\n SCPU_USAGE $P_USED $P_SHR $TOTAL" > deviceUsage.txt  
    done  
    echo $CPU_USAGE  
}
```

Sample 1

Wait

Sample 2

Result

0x46) Implementare – shell

❑ MECANICA DE STALL

- ❑ Ținând cont că avem nevoie de un sleep ca să măsurăm utilizarea procentuală a procesorului, vom folosi funcția și ca un delay între interacțiunile buclei infinite (funcția este apelată la fiecare iterație)
- ❑ Putem să adăugăm o mecanică de STALL, adică să oprim script-ul la un time-frame anume, creând un fișier temporar stall.txt care să anunțe ambele script-uri că le oprim momentan
- ❑ Deci realizăm mai multe prompt-uri cu 1 caracter (adică la apăsarea unei taste oarecare) cu timeout de 0.25s de 12 ori, dacă prompt-ul a avut succes (exit-code=0), creăm fișierul și mai dăm un prompt care va înceta rularea până la Enter, după care va șterge fișierul stall.txt

```
#sleep 3
for i in 1-12
do
    read -t 0.25 -n 1 key && touch stall.txt && read -p "| Program stalled! Press Enter to continue: " && rm stall.txt
done
read -rs -t 12 user nice system idle iowait irq softirq x0 x1 x2 < /proc/stat
```

0x47) Implementare – shell

- ❑ Definim o funcție care modifică valoarea “nice” a unui proces, un fel de prioritate atribuită de kernel pentru procesor (cu cât nice e mai mare, cu atât are prioritate mai mică) – o vom folosi pentru procese care trec de praguri pentru processor
- ❑ Folosim comanda renice [nice_level] [PID]
- ❑ Definim **while loop-ul** care va rula până în momentul închiderii script-ului shell
- ❑ Aici scriem în fișierul “report.txt” output-ul comenzii top, același fișier din care programul Python preia informațiile despre procese
- ❑ Apelăm funcția getDeviceUsage

```
USER=$(whoami)

reNI() {
    currentNI=$( ps -p $1 -o ni | tail -1 )
    if [[ $currentNI == "-" ]]
    then
        currentNI=0
    fi
    ni=$(( $currentNI + 3 ))
    if (( ni > 19 ))
    then
        kill $1
    else
        renice $ni $1
    fi
}

while [[ 0==0 ]]
do
    top -b -o $SORTBY -n 1 | tail -n +7 > report.txt
    #ps -eo pid,ni,user,rss,%cpu,time,comm --sort=-$SORTBY > report.txt
    cpuUsage=$(getDeviceUsage)
```

0x48) Implementare – shell

- ❑ Intrăm într-un alt while care citește fișierul “report.txt” linie cu linie (fără linia 1 care reprezintă field-urile) și extrage valorile respective **PID**-ului, **USER**-ului care a executat comanda, memoria **RAM** folosită (în KB), utilizarea **CPU** (procent), **numele** comenzii, etc.
- ❑ Excludem din aceste intrări procesele utilizatorului **root**; chiar dacă script-ul este rulat cu sudo, **nu** se vor închide procesele create de root

```
tail +2 report.txt |
while read pid usr pr ni virt ram shr S cpu mem t name
do
    if [[ $usr != "root" ]]
    then
        IGNORE=0
        while read -r ignoreElem
        do
            if echo $name | grep -q $ignoreElem
            then
                IGNORE=1
            fi
        done < whitelist.txt
        for ignoreElem in $keywords
        do
            if echo $name | grep -q $ignoreElem
            then
                IGNORE=1
            fi
        done
        if (( IGNORE == 0 ))
        then
```

0x49) Implementare – shell

- ❑ Identificarea proceselor care se află în fișierul “whitelist.txt” ; nu le verificăm pentru trecerea pragurilor
- ❑ Citim linie cu linie procesele din fișier și folosim grep pentru a găsi un match cu numele procesului
- ❑ Dacă IGNORE este 0, continuăm cu verificările pentru ramLimitMB și cpuLimit
- ❑ Fișierul “whitelist.txt” nu este modificat de script-ul shell, ci de utilizator
- ❑ *keywords este un fel de whitelist **local**, definit când executăm script-ul

```
if [[ $usr != "root" ]]
then
    IGNORE=0
    while read -r ignoreElem
    do
        if echo $name | grep -q $ignoreElem
        then
            IGNORE=1
        fi
    done < whitelist.txt
    for ignoreElem in $keywords
    do
        if echo $name | grep -q $ignoreElem
        then
            IGNORE=1
        fi
    done
    if (( IGNORE == 0 ))
    then
```

0x4A) Implementare – shell

- ❑ **Condiția if pentru depășirea pragului de memorie (rezidentă / fizică)**
 - ❑ Folosim variabila KB, definită la început ca (ramLimit << 10)
 - ❑ (trasformare MB -> KB)
 - ❑ Folosim comanda kill cu semnalul default (adică -15 ⇔ SIGTERM)
 - ❑ De ce nu SIGSTOP? Pentru că SIGSTOP doar oprește procesul, iar resursele RAM sunt în continuare alocate procesului
 - ❑ De ce nu SIGKILL? Pentru că SIGKILL “omoară” abrupt procesul, și putem întâmpina erori

```
if (( ram > KB ))
then
    MBram=$(( ram >> 10 ))
    testPID $pid $name && (
        echo -e "$r Process $name (PID = $pid) RAM usage exceded $ramLimitMB MB (USED: $MBram MB) $en"
        echo -e "$r Process $pid will be stopped (SIGTERM) $en"
        kill $pid
    )
fi
```

0x4B) Implementare – shell

- ❑ **Condiția if pentru depășirea pragului procesorului**
 - ❑ Pentru a reduce eventualele erori, am ales să tratez procentul ca un **int**
 - ❑ Ștergem punctul care delimitează partea fracționară de cea întreagă și împărțim la 10
 - ❑ De data asta folosim comanda kill cu semnalul -19 (SIGSTOP), pentru că vrem ca procesul să continue după ce operațiile făcute de CPU scad
 - ❑ Pentru continuarea procesului, punem un subshell în background care folosește comanda kill -18 (SIGCONT) după un interval de timp (vedem imediat)

```
cpu=$(echo $cpu | awk '{print int($1)}' )
if (( cpu > cpuLimit ))
then
    testPID $pid $name || continue
    echo -e "$r Process $name (PID = $pid) CPU usage exceded $cpuLimit% (Used: $cpu%) $en"
    echo -e "$r Process $pid will be stopped (SIGSTOP) $en"
    kill -19 $pid
    ( // Începem un subshell în background //
```

0x4B) Implementare – shell

❑ Condiția if pentru continuarea procesului (SIGCONT)

- ❑ Atribuim fiecărui proces o condiție “pedeapsă”, procesul va trebui să aștepte cel puțin un număr egal de secunde cu valoarea nice a sa, adică între 0 și 24 de secunde (i merge din 3 în 3)
- ❑ Cu timpul, valoarea nice va crește dacă mai trece pragul odată, primind apoi o pedeapsă mai mare
- ❑ Impunem și o limită superioară de 50% pentru utilizarea maximă CPU

```
kill -19 $pid
(
  i=0
  while (( cpuUsage > 50 && i < 24 || i < $ni ))
  do
    sleep 3;
    i=$(( i + 3 ))
  done
  kill -18 $pid
  reNI $pid
  echo -e "$g Process $name (PID = $pid) running after waiting $i seconds... (SIGCONT) $en"&
```

Loop-ul de așteptare

Continuăm procesul dar creștem nice value

Subshell în background

0x50) Implementare UI - python

- ❑ Vrem să implementăm o tabelă care să afișeze un anumit număr de procese și informațiile lor organizate pe coloane
- ❑ Fiecare celulă are un text box care conține valoarea procesului din coloana i pentru fieldul de pe coloana j
- ❑ Informațiile le extragem din fișierele report.txt și deviceUsage.txt
 - ❑ Procesele din report.txt sunt sortate după RAM/CPU usage
- ❑ Ulterior, vom adăuga un text box pentru utilizarea RAM și CPU totală, dar și memoria shared
- ❑ În final, vrem să realizăm un grafic care monitorizează utilizarea RAM și CPU totală
 - ❑ Acesta va fi reprezentat de mai multe dreptunghiuri (sau trapezuri) aranjate una după alta cu înălțimea direct proporțională cu procentul resursei folosite

0x51) Implementare UI - python

```
import time
from graphics import *

def main():

    n = 50

    height = 20 #inaltimea unui dreptunghi
    windowHeight = height*n

    win = GraphWin("// PROCESSES //", 800, windowHeight)
    win.setBackground(color_rgb(0,0,0))

    def makeRectangle(x1,y1,x2,y2,*clr):
        rect = Rectangle(Point(x1,y1), Point(x2,y2))
        rect.setFill(color_rgb(0,0,0))
        if clr:
            rect.setFill(clr)
        rect.setOutline(color_rgb(0,255,0))
        if clr:
            rect.setOutline(clr)
        rect.setWidth(2)
        return rect
```

Doar n elemente UI de
tip proces sunt afișate

Înălțimea în pixeli ai
unui element UI de
tip proces

Crearea unui window

Point(x1,y1)

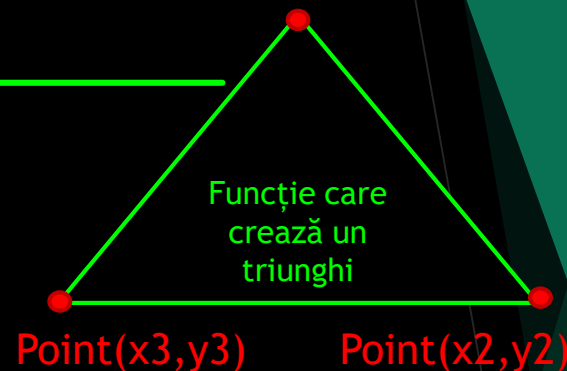
Funcție pentru crearea
unui dreptunghi

Point(x2,y2)

0x52) Implementare UI - python

```
def makeTriangle(x1,y1,x2,y2,x3,y3,*clr):  
    tri = Polygon(Point(x1,y1),Point(x2,y2),Point(x3,y3))  
    tri.setFill(color_rgb(0,0,0))  
    if clr:  
        tri.setFill(clr)  
    tri.setOutline(color_rgb(0,255,0))  
    if clr:  
        tri.setOutline(clr)  
    tri.setWidth(2)  
    return tri  
  
pbcpu = makeRectangle(20,20,21,20+height,color_rgb(0,255,0)) #progress bar pt cpu  
pbcpu.draw(win)  
  
pbram = makeRectangle(420,20,421,20+height,color_rgb(0,255,0)) #progress bar pt ram (used)  
pbram.draw(win)  
  
pbram1 = makeRectangle(420,20,421,20+height,color_rgb(255,255,0)) #progress bar pt ram (shared)  
pbram1.draw(win)  
  
headerList = [] # 0 - cpu used text ; 1 - ram used text  
chartLIST = []  
chartRECT = []  
textList = []  
sizeX = [100,100,125,75,200,200]  
header = ["PID", "USER", "RAM", "%CPU", "TIME", "COMMAND"]  
  
chartENTRIES=30
```

Point(x1,y1)



26.1% CPU USED

46% RAM USED | 6% RAM SHARED | 3964MB TOTAL

PID	USER	RAM	%CPU	TIME	COMMAND
-----	------	-----	------	------	---------

0x53) Implementare UI - python

```
def updateText(file_in,usage_file_in,win : GraphWin):  
    with open(file_in,"r") as f:  
        for l,linie in enumerate(f):  
            if l > 0 and l < n-1:  
                for i,info in enumerate( linie.strip().split() ):  
                    ramUsed=0  
                    cpuUsed=0  
                    if i==2:  
                        ramUsed = int(info) >> 10  
                        info = str(int(info) >> 10) + " MB"  
                    if i==3:  
                        cpuUsed = float(info)  
                    textList[(l)*6+i].setText(info)
```

Deschidem
fișierul report.txt

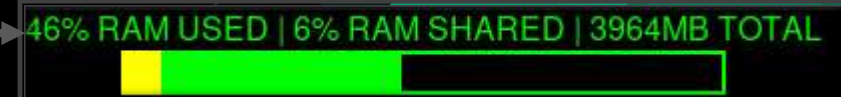
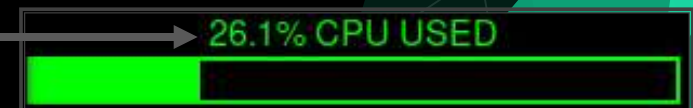
PID	USER	RAM
1652	andrei	273 MB

... textList[0]
... textList[1]
...

```
valuesLIST = []  
with open(usage_file_in,"r") as uf:  
    for l,linie in enumerate(uf):  
        values = linie.strip().split()  
        if l > 0:  
            valuesLIST.append(values)  
if len(valuesLIST)>0:  
    valuesEnd = valuesLIST[0]  
    headerList[0].setText(valuesEnd[0] + "% CPU USED")  
    headerList[1].setText(valuesEnd[1] + "% RAM USED | " + valuesEnd[2] + "% RAM SHARED")  
    return [valuesEnd[0],valuesEnd[1],valuesEnd[2]]  
return []
```

Deschidem fișierul
deviceUsage.txt

[%CPU, %RES, %SHR]



0x54) Implementare UI - python

```
def addChartElement(sizeOfCol, chartLen):  
    xstart = chartLen * sizeOfCol  
    p_cpu = int(float(ret[0]))  
    p_ram = int(float(ret[1]))  
    p_ram1 = int(float(ret[2]))  
    cpu_rect = makeRectangle(20 + xstart, 100 - p_cpu + 45, 20 + xstart + sizeOfCol, 145, color_rgb(0, 255, 0))  
    ram_rect = makeRectangle(420 + xstart, 100 - p_ram + 45, 420 + xstart + sizeOfCol, 145, color_rgb(0, 255, 0))  
    ram1_rect = makeRectangle(420 + xstart, 100 - p_ram1 + 45, 420 + xstart + sizeOfCol, 145, color_rgb(255, 255, 0))  
    cpu_rect.draw(win)  
    ram_rect.draw(win)  
    ram1_rect.draw(win)  
  
    chartRECT.append([cpu_rect, ram_rect, ram1_rect])  
  
def intify(x):  
    return int(float(x))
```

p_ram

p_ram1

20 + xstart

Un dreptunghi
din grafic

ram_rectangle

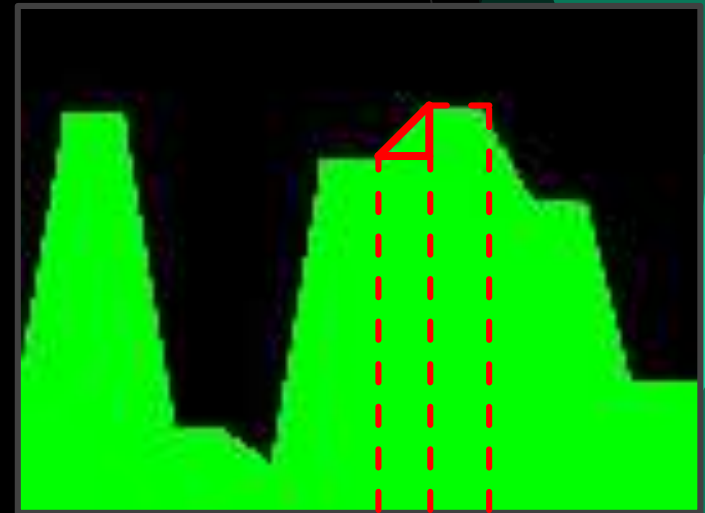
ram1_rectangle

sizeOfCol

0x55) Implementare UI - python

```
if chartLen > 0:
    lastElem=chartLIST[len(chartLIST)-1]
    y0=100-p_cpu+45
    y1=100-p_ram+45
    y2=100-p_raml+45
    dy0=100-intify(lastElem[0])+45
    dy1=100-intify(lastElem[1])+45
    dy2=100-intify(lastElem[2])+45
    if dy0 > y0:
        cpu_tri=makeTriangle(20+(xstart-sizeOfCol),dy0,20+xstart, y0, 20+xstart, dy0,color_rgb(0,255,0))
        cpu_tri.draw(win)
        chartRECT[len(chartRECT)-2].append(cpu_tri)
    else:
        cpu_tri=makeTriangle(20+xstart,dy0,20+(xstart+sizeOfCol), y0, 20+xstart, y0,color_rgb(0,255,0))
        cpu_tri.draw(win)
        chartRECT[len(chartRECT)-1].append(cpu_tri)
    if dy1 > y1:
        ram_tri=makeTriangle(420+(xstart-sizeOfCol),dy1,420+xstart, y1, 420+xstart, dy1,color_rgb(0,255,0))
        ram_tri.draw(win)
        chartRECT[len(chartRECT)-2].append(ram_tri)
    else:
        ram_tri=makeTriangle(420+xstart,dy1,420+(xstart+sizeOfCol), y1, 420+xstart, y1,color_rgb(0,255,0))
        ram_tri.draw(win)
        chartRECT[len(chartRECT)-1].append(ram_tri)
    if dy2 > y2:
        raml_tri=makeTriangle(420+(xstart-sizeOfCol),dy2,420+xstart, y2, 420+xstart, dy2,color_rgb(255,255,0))
        raml_tri.draw(win)
        chartRECT[len(chartRECT)-2].append(raml_tri)
    else:
        raml_tri=makeTriangle(420+xstart,y2,420+(xstart+sizeOfCol), y2, 420+xstart, y2,color_rgb(255,255,0))
        raml_tri.draw(win)
        chartRECT[len(chartRECT)-1].append(raml_tri)
return chartRECT
```

Implementează
triunghiurile care
conectează
dreptunghiurile vecine



0x56) Implementare UI - python

```
makeRectangle(20,20,320,20+height).draw(win) #bounding box pt cpu
tcpu = makeText(20+300/2, height/2, "%%", 12)
tcpu.draw(win)
headerList.append(tcpu)

makeRectangle(420,20,720,20+height).draw(win) #bounding box pt ram
tram = makeText(420+300/2, height/2, "%%", 12)
tram.draw(win)
headerList.append(tram)

makeRectangle(20,45,320,145).draw(win) #bounding box pt grafic cpu
makeRectangle(420,45,720,145).draw(win) #bounding box pt grafic ram

for y1 in range(9,n+9):
    x1 = 0
    for width in sizeX:
        makeRectangle(x1, y1*height, x1+width, (y1+1)*height).draw(win)

        text = makeText(x1+width/2, y1*height+height/2,"???",12)
        text.draw(win)
        textList.append( text )
        x1 += width

for i in range(0,6):
    textList[i].setText(header[i])
```

Inițializarea

- Creează “bounding boxes” pentru progress bar-uri și grafice
- Creează elementele UI de tip proces și le adaugă în lista **textList**
- **Setează header-ul** listei de procese (adică textList[0])

0x57) Implementare UI - python

```
while 1:
    ret = updateText("report.txt", "deviceUsage.txt", win)
    if len(ret) > 0:
        x0=ret[0]
        x1=ret[1]
        x2=ret[2]
        newpbcpu = makeRectangle(20,20,20+int(3*float(x0)),40,color_rgb(0,255,0))
        newpbcpu.draw(win)

        newpbram = makeRectangle(420,20,420+int(3*float(x1)),40,color_rgb(0,255,0))
        newpbram.draw(win)

        newpbram1 = makeRectangle(420,20,420+int(3*float(x2)),40,color_rgb(255,255,0))
        newpbram1.draw(win)

        pbcpu.undraw()
        pbram.undraw()
        pbram1.undraw()

        pbcpu=newpbcpu
        pbram=newpbram
        pbram1=newpbram1
```

While loop

- Updatează textul elementelor UI de tip proces prin apelul funcției **updateText**
- Reconfigurează progress bar-urile prin ștergerea dreptunghiurilor anterioare și crearea lor din nou cu valorile recente

0x58) Implementare UI - python

```
sizeOfCol=int(300/chartENTRIES)
chartLen=len(chartLIST)
if chartLen < chartENTRIES:
    #adaugam dreptunghi la final
    chartRECT=addChartElement(sizeOfCol,chartLen)
else:
    #elim primul, mutam dreptunghiurile la stanga si adaugam dreptunghi la final
    firstEntry=chartRECT[0]
    chartRECT.pop(0)
    for entry in chartRECT:
        for geom in entry:
            geom.move(-sizeOfCol,0)
    chartRECT=addChartElement(sizeOfCol,chartLen-1)
    for geom in firstEntry:
        geom.undraw()

chartLen = len(chartLIST)-1
if chartLen < chartENTRIES-1:
    chartLIST.append(ret) #30 valori
else:
    chartLIST.pop(0)
    chartLIST.append(ret)

time.sleep(1.25)
```

While loop

- Adaugă intrări în graficele pentru monitorizarea CPU și RAM
- Dacă s-au depășit numărul prestabilit de intrări (aici 30), trebuie să mutăm toate intrările la stânga și să eliminăm prima intrare
- În final, adăugăm un delay între iterații

0x60) Idei de îmbunătățiri

- ❑ UI interactiv
- ❑ Comunicare mai bună între script-uri (nu prin fișiere)
- ❑ Sincronizarea script-urilor (poate folosind epoch time)
- ❑ Implementarea unui systemd .device file care să încorporeze script-ul shell
- ❑ Mai multe funcționalități user-friendly



0x70) Bibliografie

❑ 0x39)

- ❑ <https://mcsp.wartburg.edu/zelle/python/>
- ❑ https://www.youtube.com/playlist?list=PLmzkEJ1Zz_uZ5nvTOLaGHfinCzEVEVBiz

❑ 0x42)

- ❑ <https://www.geeksforgeeks.org/linux-unix/top-command-in-linux-with-examples/>
- ❑ https://www.gnu.org/software/bash/manual/html_node/Shell-Arithmetic.html

❑ 0x45)

- ❑ <https://share.google/nH6EQiJM77jqR6aXH>
- ❑ <https://share.google/c75SVlozqYjizvHXM>

❑ 0x46)

- ❑ <https://www.idnt.net/en-GB/kb/941772>
- ❑ <https://www.linuxhowtos.org/System/procstat.htm>

❑ 0x47)

- ❑ <https://securityzines.com/pages/flyers/lproc>

<https://github.com/AndreiCd27/Watchdog-ITBI>